2. Oracle OCI — System Design

> Loop only (not Round 1). 1–2 rounds of LLD and/or HLD, iterative requirement additions, heavy tradeoff probing. Full answers: `sd##.problem-name.md` files in this folder. This file = question bank + coverage map + drafts for everything without a file yet. Infra-algorithm code drills (LRU, rate limiter, KV cache, Kafka log — Python+Java, v1 + follow-up): 2.sd1.md. Bank rows 1–9 (infra) + 19–24 (consumer) = **first-hand 1p3a reports (P0)** — details in `../1.coding/0.1p3a.md`; rows 10–18 = distributed/platform core; rows 25+ = rest. JD + recruiter docs: `../0.req/` (JD = PKI team; recruiter sheet = process + scoring; prep pack = core values). Rules: `../../AGENTS.md` (26U).

Index

---

1. How Oracle runs SD rounds

```
"Design X" (vague) → you scope → simple v1 → +requirement → scale → +requirement → tradeoffs
```

- Start simple, iterate → rewarded explicitly: "grungy solution now" (Act now, iterate)
- Vague on purpose → they score the questions you ask
- LLD prompts → API + schema + class layout + concurrency in one round
- Always two approaches ready for any locking/consistency question
- HM round → draw *your* JPMC system, defend under "why not Y"
- SD 可能塞在 BQ/behavior 轮的最后 10 分钟 (1p3a) → 备一个 5 分钟 skeleton 开场: scope 一句 + 3 个盒子 + 一个 tradeoff
- HM 轮的 SD 可能变成纯项目聊天 (1p3a) → 随时能画自己的 JPMC 系统并守住 "why X not Y"
- Managerial 轮设计完还要交付文档 (1p3a) → **EoE (Estimation of Effort) + rough spec document** — 设计之外考交付物
- 时间管理是最常见失分点 (1p3a) → 每块限时; 宁可每块浅而完整, 不要一块深而讲不完

2. Question bank — ranked

Legend: 1p3a = first-hand report · ★★★ multiple candidate reports · ★★ two · ★ one · ◇ unsourced aggregator only · 🎯 fits JD/team Rows 1–9 = first-hand infra (P0, 前置); rows 10–18 = distributed/platform core (全部有 full file 或 60-s 答案); rows 19–24 = first-hand 消费级; rows 25+ = 其余。

#	Prompt	Evidence	Type	Priority
1	Rate limiter (distributed) →sd01	1p3a ×2 + recruiter + ◇	HLD + algo	🎯 P0
2	URL shortener / bitly →sd02	1p3a + ★★★	HLD	P0 (easy)
3	Cache system: thread-safe LRU (LLD) → distributed cache →sd03	1p3a ×2 + ★★★ + ◇	LLD + HLD	🎯 P0
4	Logging system / log file system →sd04	1p3a + ★ + ◇	LLD	P0
5	Healthcare data ingestion pipeline →sd05 §12	1p3a	HLD	P0
6	Software load balancer (L4 / L7) →sd06	1p3a	HLD	🎯 P0
7	Internal metric ingestion service →sd05	1p3a	HLD	🎯 P0
8	Edge-device health monitoring @ millions →sd08	1p3a	HLD	🎯 P0
9	Scale a monolithic service	1p3a	HLD discussion	P0
10	Distributed job scheduler / orchestrator →sd10	★★★	HLD + concurrency	🎯 P0
11	Distributed KV store →sd11	Alex Xu + ◇	HLD	P0
12	Distributed lock manager (lease, fencing) →sd12	◇	LLD/HLD	🎯 P1
13	Certificate management service →sd13	team domain	HLD	🎯 P1
14	Control-plane work-request API →sd14	JD	HLD	🎯 P1
15	Kafka-like distributed log →sd15	◇	HLD	🎯 P1
16	Thread-safe DB connection pool →sd16	◇	LLD	🎯 P1
17	Distributed file / object storage →sd17	◇	HLD	🎯 P1
18	Distributed transaction coordinator (2PC)	◇ (DB group)	HLD	discussion only
19	Video streaming (YouTube/Netflix, global)	1p3a	HLD	P0
20	Remote Browser Isolation (enterprise)	1p3a	HLD	P0
21	E-commerce recommendation system	1p3a	HLD	P0
22	大文件上传容错 (chunk / resume) →sd22	1p3a	HLD component	P0
23	Chat app (WhatsApp/Slack, 1B / 10M DAU)	1p3a	HLD	P0
24	Design Uber (matching, location, trip)	1p3a ×2	HLD	P0 (消费级)
25	Ticket booking / BookMyShow / flash sale	★★ + ◇	LLD→HLD, locking	P1
26	E-commerce / cart	★★	HLD	P2
27	Typeahead (trie, top-k)	◇	HLD	P2
28	Proximity search (geohash/quadtree)	◇	HLD	P3
29	Meeting-room booking; door access control	★	LLD	P3
30	Backup + point-in-time restore	◇ (DB group)	HLD	discussion only

#	Prompt	Evidence	Type	Priority
31	B-tree index class layout	◇ (DB group)	LLD	discussion only

First-hand notes (rows 1–9 + 19–24, from the reports):

1. Rate limiter — 讨论: 不同算法对比、caching、distributed implementation、scalability + trade-offs
2. URL shortener — 轮末最后 10 分钟才出题, 无大问题
3. Cache system — 一轮: 设计 LRU 并解释数据结构选型; 另一轮: design a cache system; LLD → 分布式两段都要能接
4. Logging file system — LLD/HLD 混合; 相关 LC 635 (`../1.coding/1.coding-1.md`)
5. Healthcare data ingestion — 数百万条记录, 8 点全要碰: ingestion / queue-broker 选型 / storage / processing / fault tolerance / scalability / monitoring+alerting / 失败记录处理
6. Software load balancer — L4 或 L7 自选; LB policies + healthchecks
7. Internal metric ingestion service — client agent 逻辑, sharding, availability, 读写关注分离, infra 选型
8. Edge-device health monitoring — 1M 设备; pull vs push 对比; 报告人答案: push 为主 + pull double-confirm
9. Scale a monolith — 拆 microservices, 按 component 找 bottleneck, 独立 scale
10. Video streaming — 给定网页 URL (schemaName, host, port) 起步; 全球分发
11. Remote Browser Isolation — authn/authz, data streaming, gateways, rate limiting, virtual nodes, edge servers
12. Recommendation system — 大规模电商, 基于用户浏览历史
13. 大文件上传容错 — chunked upload + 每 chunk 校验/重传 + 断点续传 + 幂等提交 + 整体 hash
14. Chat app — 1B users / 10M DAU; API design, DB 选型, guaranteed message delivery (ack + 持久化 + 重试/幂等)
15. Design Uber — driver matching, real-time location tracking, trip management; 面试官深挖 scalability / bottleneck / 分布式; 教训: 控制每块深度, 别讲不完

附: 另有报告含未具体化的 SD 讨论; 架构八股 (SQL vs NoSQL, queue 取舍, SLA, indexing, 分层 scaling) 见 `../1.coding/0.1p3a.md` §1.2。

3. What interviewers probe

- Scoping → FRs, non-goals, the scale numbers you *ask for*
- Data + API → schema; REST idempotence; pagination; versioning
- Caching → invalidation; stampede; TTL by freshness class
- Concurrency → optimistic vs pessimistic; lease vs lock; fencing tokens → "give me a second approach"
- Consistency → model per store; what breaks under partition
- Security → OAuth2, mTLS, per-tenant authz, audit, compartments
- Scale → bottleneck; "what breaks at 10×"; cost
- Platform/control-plane specific → CP vs DP separation; static stability → blast radius: FD / AD / region; cells; canaries → leader election where one writer needed → ops readiness: dashboards, alarms, runbook, rollback
- The "why" → SQL vs NoSQL; which cache; which queue — defend with the access pattern

4. Coverage map — Alex Xu vol 1 vs gaps

- Vol 1 covers (finish by Sun) → rate limiter · consistent hashing · KV store · unique ID · URL shortener → crawler · notification · news feed · chat · autocomplete (typeahead) · YouTube · Drive (file storage) → 1p3a P0 hits directly: rate limiter, URL shortener, chat, YouTube/streaming
- Full files now cover the distributed/platform set (2026-08-31): sd10–sd02 + **sd06 LB · sd05 ingestion (metrics + healthcare) · sd08 edge health · sd04 logging FS · sd15 Kafka log · sd16 connection pool · sd17 object storage · sd22 file upload**
- Still draft-only → use §11: monolith scaling, video streaming, RBI, recommendation, chat, Uber, 2PC, ticket booking, cart, typeahead, proximity, meeting-room, backup, B-tree
- Speed rule → book chapters: read → §16 card only → full `Generate #N`: §5 list only

5. Team-aligned designs — do fully

1. Distributed job scheduler — `sd10.job-scheduler.md` → crux: at-least-once delivery + idempotent execution; leader per shard; lease + fencing → anchor: SQS/Lambda + idempotency keys
2. Rate limiter — `sd01.rate-limiter.md` → token bucket vs sliding window; Redis + Lua; local + global tiers
3. Certificate management service — `sd13.certificate-management.md` → CP: strong consistency, HSM-signed, audited · DP: LB caches cert, survives CP outage
4. Distributed lock manager — `sd12.distributed-lock.md` → lease with TTL; fencing token monotonic; ZooKeeper/etcd vs Redis Redlock
5. Control-plane work-request API — `sd14.work-request-api.md` → 202 + work-request id; state machine; idempotent retries; per-tenant quotas; audit
6. Distributed KV store — `sd11.kv-store.md` → consistent hashing; leader vs leaderless replication; quorum R+W>N; anti-entropy
7. URL shortener — quick — `sd02.url-shortener.md`
8. First-hand / infra set (2026-08-31): `sd06` LB · `sd05` ingestion pipeline · `sd08` edge health · `sd04` logging FS · `sd15` Kafka-like log · `sd16` connection pool · `sd17` object storage · `sd22` file upload

6. LLD / machine-coding bank (30 min each: classes + core methods + locks)

- LRU / LFU cache, thread-safe — also Round 1 coding; 1p3a 有轮要求 "explain the DS choice"
- DB connection pool → `sd16.connection-pool.md`
- Distributed lock client → acquire(lease) → token; heartbeat renew; release; fencing check
- Ticket booking → seat hold with TTL; optimistic version column; state machine HELD→BOOKED
- Rate limiter class (token bucket, sliding window)
- Access-control system — roles, revocation, audit → your RBAC work
- Logging system — append, rotate, query by time range → `sd04.logging-file-system.md`
- Meeting-room booking — interval conflicts
- Replicated active–passive hashmap; concurrent task processor — first-hand LLD coding, solved in `../1.coding/0.1p3a.md` §1.3

7. Discussion-level DB internals (60 s each, no full design)

- MVCC → readers see snapshot version; writers create new version; no read locks → defeater: write skew under snapshot isolation; vacuum/undo growth
- B-tree index → balanced, high fan-out, leaf-linked; O(log n) lookup; range scans → defeater: random inserts → page splits; write amplification vs LSM
- 2PC → prepare → commit; coordinator log → defeater: blocking on coordinator crash; prefer saga/outbox
- Replication → sync vs async; log shipping; RPO/RTO
- Point-in-time restore → full backup + WAL/redo replay to timestamp
- Execution plan → index vs seq scan; join order; stats freshness

8. Your JPMC anchors

- schema evolution + Liquibase drift tests → safe contract change
- at-least-once SNS/SQS + idempotency → scheduler, work requests
- Aurora canonical + Mongo read model (CQRS) → cart / read-heavy
- RBAC resolution + audit → access control, IAM
- Datadog/CloudWatch RCA → ops-readiness section every time
- Lambda test injector → "how did you test it"

9. Unsourced aggregator list — kept for cross-check

- Source: AI-generated summary, no citations, mixes OCI / Database / Fusion groups
- Claims → HLD infra: global KV store; Kafka-like log; backup + PIT restore; distributed transaction coordinator; global file storage → HLD business: flash sale / Ticketmaster; Yelp proximity; typeahead → LLD: LRU/LFU thread-safe; connection pool; distributed lock manager; distributed rate limiter; B-tree index → themes: defend the "why"; DB internals (replication, plans, MVCC); 99.99% + compartments + tenant isolation
- Assessment → agrees with sourced reports on: KV, rate limiter, cache, booking → plausible for platform team: lock manager, connection pool, log, object storage — all now have full files (sd12, sd16, sd15, sd17) → DB-group flavored, keep discussion-only: B-tree, PIT restore, 2PC, GoldenGate → treat "themes" as accurate — they match recruiter's term list
- Action (partially done) → 1p3a cross-check 2026-08-31: rate limiter / cache / logging confirmed first-hand → promoted to P0 → keep verifying against RedNote 2025–26 in loop-prep week 2

11. Drafts — every bank question without a full file

Format per `26U/AGENTS.md`: crux · HLD line · data/API hint · reuse pointer · defeater. Promote to `sd##` when priority rises. Numbers = §2 table rows.

#3 Cache system — distributed-cache half (P0; LRU half = sd03)

- Crux: cache tier as a *service* — placement, invalidation, and what happens when a cache node dies (answer: nothing durable was there)
- HLD: client library → consistent-hash ring of cache nodes (sd11 §10.1 ring, no replication or N=2 async) → backing store on miss
- Data: values with TTL + version; no durability promises; `mc:{tenant}:{key}` namespacing

- Reuse: ring from sd11; stampede/ `getOrLoad` + two-tier from sd03 §10.4; invalidation via pub/sub
- Defeater: "replicate cache nodes for durability" — a cache that must not lose data is a KV store; call it what it is (sd11)

#9 Scale a monolithic service (P0, discussion)

- 60-s 骨架: 先测量找瓶颈 → 垂直扩容/缓存 (最便宜) → 读写分离 → 按 bounded context 拆 microservices (先拆最热/变更最快的) → 每个 component 独立 scale + 独立部署; 拆分代价: 分布式事务→saga, 联查→API 组合, 运维复杂度
- Defeater: "直接全拆" — strangler fig 渐进拆, 先拆边缘再拆核心
- 报告注: 就是拆分叙事 + 每个 component 的 bottleneck 怎么解

#18 Distributed transaction coordinator / 2PC (discussion only)

- 60 s answer: prepare (all vote, persist redo) → commit; coordinator log is truth; blocks on coordinator crash between phases — participants hold locks → prefer sagas/outbox (sd14 §9.2, sd10 §10.2) for cross-service; 2PC only intra-trust-domain (XA, spanner-style with Paxos coordinator)
- Defeater: "2PC gives exactly-once across services" — it gives atomicity while blocking; availability is the price

#19 Video streaming global (P0)

- Crux: 上传/转码 (异步流水线) 与播放 (CDN 边缘) 完全分离; 全球分发 = CDN + 区域源站
- HLD: upload (chunked, sd22 的模式) → 转码集群 (多分辨率 HLS/DASH 段) → object storage (sd17) → CDN; 播放走 manifest + 自适应码率; 元数据/推荐单独服务
- Reuse: Alex Xu vol 1 YouTube 章; upload 容错 = sd22
- Defeater: "源站直接服务视频" — 带宽成本与延迟; CDN 命中率是第一指标
- 报告注: 题面从 URL (schemaName, host, port) 起步 — 先把解析/路由讲清再进入流媒体

#20 Remote Browser Isolation (P0)

- Crux: 浏览发生在远端隔离容器, 终端只收像素/DOM 流 — 安全边界在哪一层
- HLD: 用户 → gateway (authn/authz, per-tenant) → 调度到 virtual node 池 (每会话一容器, 用完即毁) → 页面渲染在容器, 像素流/重构 DOM 经 edge server 低延迟回传; rate limiting + 会话录制审计
- Reuse: gateway/quota = sd01; 租户隔离/审计 = sd13-07 词汇; edge = CDN 思路
- Defeater: "所有用户共享浏览器进程" — 逃逸即全租户沦陷; 每会话隔离 + 最小权限
- 报告注: 题面点名 authn/authz/data streaming/gateways/rate limiting/virtual nodes/edge — 按点覆盖

#21 E-commerce recommendation system (P0)

- Crux: 离线/近线/在线三层 — 浏览历史进流, 模型离线训, 在线只做召回+排序
- HLD: 行为事件 → Kafka (sd15) → 特征存储; 离线训练 (协同过滤/双塔) 产出 embedding + 候选索引; 在线: 召回 (ANN/规则) → 排序 (轻模型) → 过滤去重; 冷启动用热门/类目
- Defeater: "在线实时训练" — 延迟与稳定性; 近线更新 embedding 已足够
- 报告注: 基于浏览历史 — 讲清事件采集 → 特征 → 召回排序链路即可

#23 Chat app 1B / 10M DAU (P0)

- Crux: guaranteed delivery = 服务端持久化 + ack + 幂等重投; 在线走长连接, 离线走收件箱+推送
- HLD: client ↔ gateway (WebSocket, 连接注册) → chat service → 消息持久化 (按会话分片) → 投递 (在线 push / 离线 inbox + notification); 已读/未读游标; 群聊扇出写 vs 扇出读
- Data: `messages(conv_id, seq, sender, body, ts)` 按 conv 分片, seq 单调; `inbox(user, conv, last_ack_seq)`
- Reuse: Alex Xu vol 1 chat 章; presence = sd08 心跳模型; 幂等 = sd14 §9.1
- Defeater: "at-least-once 就是 exactly-once" — 客户端按 (conv, seq) 去重才成立; 说清 ack 链路 (client→server→storage→ack)
- 报告注: 题面点名 API design / DB choices / guaranteed delivery — delivery 讲最细

#24 Design Uber (P0, 消费级)

- Crux: driver-rider matching on live location — geo index that updates every few seconds without melting
- HLD: rider request → matching service (geo lookup: geohash/quadtree cells, #28's index) → nearest available drivers → offer/accept state machine → trip service (state, fare); locations via driver-app heartbeats into an in-memory geo store (Redis GEO / cell → driver set)
- Data: `drivers(id, cell, status, last_seen)` hot in memory; `trips(id, rider, driver, state, route…)` durable; location history to a stream for ETA/ML, not the hot path
- Reuse: heartbeat + presence = sd08 push 模型; surge/quota = sd01 thinking
- Defeater: "store live locations in SQL" — 10^6 drivers × 4 s updates is a write firehose; memory + stream, SQL only for durable trip state
- 报告注: 面试官盯 scalability/bottleneck; 按 matching → location → trip 分块讲, 每块给瓶颈和 10× 方案

#25 Ticket booking / flash sale (P1)

- Crux: N users, 1 seat — contention on a tiny hot set; oversell = incident, undersell = lost money
- HLD: browse (cached, stale-ok) → hold seat (TTL lease, sd12 §8 row-lease pattern: `UPDATE seats SET state='HELD', hold_until=... WHERE seat_id=? AND state='FREE'`) → pay → confirm (state machine HELD→BOOKED, fence = version column)
- Data: `seats(event_id, seat_id, state, version, hold_until, order_id)`; partial index on FREE; holds expire by reaper
- API: `POST /events/{id}/holds {seat_ids}` → 200 hold_id + expires | 409; `POST /holds/{id}/confirm` idempotent by hold_id
- Flash-sale layer: queue + admission (virtual waiting room) in front; inventory counter in Redis `DECR` with floor-at-zero Lua (sd01 script pattern) as the fast gate, DB as truth
- Defeater: "lock the whole event row" — serializes all seats; lock the *seat*, or shard inventory counters

#26 E-commerce / cart (P2)

- Crux: cart = per-user low-contention writes (easy) vs checkout = inventory + payment consistency (the actual question — steer there)
- HLD: cart in KV (sd11; LWW-with-merge: union items, max qty — say the merge rule) → checkout = saga: reserve inventory → charge → confirm, compensations in reverse (sd14 §9.2)
- Data: `carts(user_id) → items jsonb` in KV; `inventory(sku, available, version)` optimistic-locked in SQL; `orders` state machine

- Reuse: idempotent charge via Idempotency-Key (sd14 §9.1); JPMC anchor: Aurora canonical + Mongo read model (CQRS) — your own story
- Defeater: "decrement inventory when adding to cart" — carts abandon; reserve only at checkout with TTL holds (#25's pattern)

#27 Typeahead (P2)

- Crux: p99 < 50 ms per keystroke; results are *precomputed*, not searched
- HLD: offline pipeline aggregates query counts → builds trie/FST with top-k cached **at each node** → immutable snapshot shipped to serving fleet (double-buffer swap); serve = walk prefix, return stored top-k
- Data: trie node = `{children, top_k: [(term, score)]}`; shard by prefix range; hot prefixes ("a") are pure cache hits
- Reuse: build/serve split = CP/DP separation (`../4.1o-arch.md` §2); snapshot swap = static stability
- Defeater: "update the trie on every query" — online updates wreck tail latency; batch rebuild (minutes-stale is fine for suggestions)

#28 Proximity search / Yelp (P3)

- Crux: "points near (lat,lng)" needs a spatial index — B-tree on lat, lng can't do circles
- HLD: geohash (prefix = cell; neighbors of cell for edge cases) or quadtree; candidates from cell + 8 neighbors → exact distance filter → rank
- Data: `places(geohash6, place_id, lat, lng, ...)` indexed on `(geohash6)`; or PostGIS/Redis GEO and say you'd buy not build
- Defeater: "geohash prefix alone" — cell-boundary neighbors missed; always query the 8 neighbors too

#29 Meeting-room booking / door access (LLD, P3)

- Crux: interval conflict detection + authz
- Core: per-room ordered set of bookings; conflict iff `new.start < existing.end AND new.end > existing.start`; insert under per-room lock or DB exclusion constraint — `EXCLUDE USING gist (room_id WITH =, tstzrange(start_t, end_t) WITH &&)` (the one-line schema answer that wins the round)
- Reuse: LC 56 Merge Intervals (1.coding-1.md merge) is the algorithmic core; access control = RBAC story from JPMC
- Defeater: "check-then-insert" — TOCTOU double-booking; the constraint or lock must be atomic with the insert

#30 Backup + point-in-time restore (discussion only)

- 60 s answer: full snapshot + continuous WAL archive; restore = latest full < T, replay WAL to T; RPO = archive lag, RTO = restore + replay time; test restores monthly or you have backups, not restore capability
- Defeater: "replicas are backups" — replicas replicate your DROP TABLE instantly

#31 B-tree index class layout (discussion only)

- 60 s answer: `Node{keys[], children[]|values[], leaf, next_leaf}`; high fan-out (page-sized nodes, ~100s of keys) → 3–4 levels for billions; leaf-linked for range scans; insert splits upward at capacity

- vs LSM (sd11 §9): B-tree = read-optimized in-place; LSM = write-optimized append; that contrast is usually the actual question

10. Sources

- First-hand: `../1.coding/0.1p3a.md` — 18 reports, 2026-08-31 pass (SD items = bank rows 1–15)
- LeetCode Discuss OCI 2021–2026 (list in `../0.1o.md` §8)
- roundz.substack.com (Aug 2025); khaniqbal.medium.com (Jul 2024); GeeksforGeeks OCI IC3
- Recruiter prep sheet; Oracle prep pack
- OCI Certificates + PKI docs (`../4.1o-arch.md` §9)
- Hello Interview: job scheduler, Ticketmaster, rate limiter, distributed lock
- Alex Xu vol 1 + vol 2; HikariCP GitHub; etcd/ZooKeeper docs; Kleppmann "How to do distributed locking"